

GlideQuery: chain rules, Stream, and Optional

This note explains why a GlideQuery chain stops, how to store a query and add to it, and which methods belong to Stream versus Optional. Names follow the official API: **orderBy** / **orderByDesc**, and **deleteMultiple**. **orderByAsc** and a GlideQuery **delete()** are not in the documented method list. In a scoped app the class is **global.GlideQuery**.

Contents

Entries link to the section. The PDF outline lists the same destinations.

Why GlideQuery, where it runs, and what it cannot do

1. The three objects
2. Basics
3. Why select().where() fails
4. Storing a query and adding to it
5. Chain rules
6. When to use each object
7. Every syntax that produces a Stream or an Optional
8. Stream versus Optional syntax
9. Writes
10. Names that cheat sheets disagree on
11. Progressive examples
12. Beside GlideRecord and GlideAggregate
13. One-to-many and many-to-many
14. Grouped sum — the object, not a method
15. Quiz — easy
16. Quiz — medium
17. Quiz — hard
18. Performance

19. Critique of this note

22. Terms, a sample set, and output quizzes

Appendix. Not GlideQuery

24. A simple count: number, Stream, Optional, GlideRecord

There is no Australia-only edition. Now Learning is one catalog; virtual instructor-led is the form available in Australia.

Scripting in ServiceNow Fundamentals (SSNF) — 3-day instructor-led. Lab Explore GlideQuery. Syllabus KB0012764.

Scripting in ServiceNow — free on-demand, Zurich. Server-side module is GlideRecord. Use it for Business Rules and Script Includes around these queries.

[GlideQuery API](#) · [Stream API](#) · [Optional API](#) · [Product docs](#)

Why GlideQuery, where it runs, and what it cannot do

GlideQuery is a server-side alternative to GlideRecord for reading and writing rows. Use it when a bad field name must throw instead of matching every row, when you want plain JavaScript values, and when the result is either a list (Stream) or one row that might be missing (Optional). It does not replace GlideRecord, GlideAggregate, or gs. Those still own the record you are sitting on, odd aggregates, and everything that is not a table query.

Why use it

A typo in GlideRecord is ignored. where('activated', true) on GlideRecord matches every row, and a following deleteMultiple can empty the table. The same call on GlideQuery throws NiceError and runs nothing. That is the reason to reach for it on a write.

Values come back as strings and numbers, not GlideElement. === works. You do not call query() or next(). A missing row is an empty Optional, not a false from get().

Simple count, sum, max, min, and avg are one call. A grouped sum is aggregate plus groupBy plus select, and the row is { group, sum }. Use GlideAggregate when you already need date-part grouping or several aggregates in the style you have.

Where it runs

Place	GlideQuery?	Use instead
Business rule, script include, scheduled job, fix script, background script	Yes. Server-side.	current and previous in a business rule are still GlideRecord.
Scripted REST, Flow script step, server-side UI action	Yes.	gs for the user, the message, and the event.
Service Portal widget server script, a Script Include called by GlideAjax	Yes, on the server half.	The client controller cannot call GlideQuery.
Client script, catalog client script, UI policy, portal client, workspace client script	No.	GlideAjax to a script include, or GlideForm. No table query in the browser.

Place	GlideQuery?	Use instead
Workspace data broker or UX server script	Yes if that script is server-side.	Client UX scripts are not.

Scoped apps must use `global.GlideQuery`. Global scripts can call `GlideQuery`. It is not a client API, and it is not available in a portal or workspace controller.

What it cannot do

You need	Why GlideQuery is the wrong tool	Use
The record this business rule is running on	current is a <code>GlideRecord</code> . <code>GlideQuery</code> cannot see previous.	current, previous. Query other rows with <code>GlideQuery</code> .
<code>setAbortAction</code> , <code>setRedirect</code> , or a UI action on current	Those methods are <code>GlideRecord</code> .	<code>GlideRecord</code> .
Attachments, events, notifications, properties	Not a query.	<code>GlideSysAttachment</code> , <code>gs.eventQueue</code> , <code>gs.getProperty</code> .
The logged-in user, a message, a date, nil check	<code>gs</code> is <code>GlideSystem</code> , not a query API.	<code>gs.getUser</code> , <code>gs.getMessage</code> , <code>GlideDateTime</code> , <code>gs.nil</code> .
A display value or a <code>GlideElement</code> method	<code>select</code> returns plain values. <code>\$DISPLAY</code> is a field flag, not <code>getDisplayValue</code> .	<code>GlideRecord.getDisplayValue</code> , or the <code>\$DISPLAY</code> flag where the release supports it.
One-row delete, or a delete that must run business rules per row	There is no single-record delete. <code>deleteMultiple</code> skips per-row script and can orphan currency rows.	<code>GlideRecord.deleteRecord</code> in a loop.
<code>MONTH()</code> grouping or several aggregates you already wrote in <code>GlideAggregate</code>	<code>GlideQuery</code> aggregate covers sum, count, min, max, avg with <code>groupBy</code> . Odd date parts stay easier in <code>GlideAggregate</code> .	<code>GlideAggregate</code> .
A join that is not a dot-walk or a child query	No <code>addJoinQuery</code> . Dot-walk or <code>flatMap</code> .	<code>GlideRecord.addJoinQuery</code> , or <code>flatMap</code> .
Anything in the browser	Server only.	<code>GlideAjax</code> to a script include that uses <code>GlideQuery</code> .

A business rule can use both. `current` is the row that fired. `GlideQuery` reads or writes other rows. `gs.info` is still how you log. A client script never calls `GlideQuery`. It calls `GlideAjax`, and the script include does.

💡 Knowledge check

Can a catalog client script call `GlideQuery`? A. Yes. B. No.

Answer. B. `GlideQuery` is server-side. The client calls `GlideAjax`; the script include runs the query.

1. The three objects

A builder is the `GlideQuery` you still hold before a terminal. `where`, `orderBy`, and `limit` return a builder. A terminal is the call that leaves it: `select`, `selectOne`, `count`, `update`. A `Stream` is the list `select` returns. An `Optional` is the one-row container. `current` is a `GlideRecord`, not a builder. `GlideQuery` is not one object with every

method on it. A script holds one of three things, and a method exists only on the object you are holding.

Object	How you get it	What you can call
GlideQuery	new GlideQuery(table), or any intermediate call	where, orWhere, whereNull, whereNotNull, orWhereNull, orWhereNotNull, orderBy, orderByDesc, limit, groupBy, aggregate, having, withAcls, disableWorkflow, forceUpdate, disableAutoSysFields
Stream	select()	map, filter, limit, flatMap (still a Stream). forEach, reduce, toArray, find (these end the Stream). find returns an Optional.
Optional	selectOne(), get(), getBy(), insert, update, insertOrUpdate, Stream.find()	isPresent, isEmpty, ifPresent, map (still an Optional). get and orElse unwrap it.

Intermediate GlideQuery methods return a new GlideQuery, so they chain. A terminal GlideQuery method leaves the builder. select returns a Stream. selectOne, get, getBy, insert, and update return an Optional. count returns a number. updateMultiple returns an object with rowCount. deleteMultiple returns nothing. parse is static: GlideQuery.parse(table, encodedQuery). It is not a method on an instance.

💡 Knowledge check

Which object has where? A. GlideQuery. B. Stream.

Answer. A. where exists only on the builder.

2. Basics

Compared with GlideRecord, you do not call query() or next(). Each builder call returns a new object instead of mutating one record pointer. Field values come back as plain JavaScript values, so a string comparison with === works. A typo in a field name throws NiceError and runs nothing. The same typo on GlideRecord is ignored, and a following deleteMultiple can match every row.

What GlideQuery is — and what it is not

A server-side fluent API for reading and writing records. You build a query, then one terminal call runs it. In a scoped app the class is **global.GlideQuery**.

COMPARED WITH GLIDERECORD

- GlideRecord mutates one object: **addQuery()**, then **query()**, then **next()**.
- GlideQuery returns a new object from each builder call. You do not call **query()** or **next()**.
- A bad field name fails fast. GlideRecord often ignores it and matches every row.
- Results are plain objects, not GlideElement. **===** works on strings.

THREE OBJECTS, NOT ONE

- **GlideQuery** — the builder. `where` `orderBy` `limit`
- **Stream** — many rows, from **select()**. `forEach` `map`
`toArray`
- **Optional** — one row or none, from **selectOne()** / **get()**.
`orElse` `isPresent`

A method exists only on the object you are holding. That is the whole chaining rule.

THE SHAPE OF A READ

```
new GlideQuery('incident')
  .where('active', true)
  .orderBy('number')
  .limit(20)
  .select('number')
  .forEach(function (row) {
    gs.info(row.number);
  });
```

Builders first. **select** hands you a Stream. **forEach** is what actually pulls rows.

STORE THE BUILDER, THEN ADD TO IT

```
var open = new GlideQuery('incident')
  .where('active', true);

var p1 = open.where('priority', 1).count();
var p2 = open.where('priority', 2).select('number');

// p2 is a Stream — no where()
p2.where('priority', 1);
```

Each **where** returns a new GlideQuery. **open** is not changed. **count()** returns a number. **select()** returns a Stream.

FAIL-FAST, BEFORE ANY WRITE

- Unknown field: **activated** instead of **active** throws. It does not match every row.
- Bad choice and wrong type throw the same way.
- **update()** needs a primary-key **where**, usually **sys_id**.
- Sets use **updateMultiple()**. Deletes use **deleteMultiple()**. There is no single-record delete.
- Run the same **where** chain through **count()** before a bulk write.

Figure 1. What GlideQuery is, how a stored builder is extended, and the fail-fast write rules.

A normal read

```
new GlideQuery('incident')
  .where('active', true)
  .orderBy('number')
  .limit(20)
  .select('number', 'short_description')
  .forEach(function (row) {
    gs.info(row.number + ' ' + row.short_description);
  });
```

where, orderBy, and limit are still a GlideQuery. select is the handoff: it returns a Stream, and sys_id is always included even if you did not ask for it. forEach is a Stream method. That is the call that pulls rows. select itself is lazy — mapping and filtering on the Stream do no work until forEach, reduce, toArray, or find.



Knowledge check

True or false: a scoped app can call new GlideQuery without a prefix.

Answer. False. Scoped scripts use global.GlideQuery.

3. Why select().where() fails

where is a method on GlideQuery. select does not return a GlideQuery, so the next call is not looking up where on the builder. It is looking up where on a Stream, and Stream has no where. The failure is a missing method, not a rejected filter.

```
new GlideQuery('incident') // GlideQuery
.where('active', true) // GlideQuery
.select('number') // Stream – builder is over
.where('priority', 1); // Stream has no where()
```

Terminal means the GlideQuery chain is over. It does not mean the database already ran. You can still map and filter the Stream. You cannot go back and add a query builder method. If the filter belongs in the query, put where before select. Stream.filter after select filters rows already headed for script, which is a different operation.

Knowledge check

select().where() fails because: A. The filter is illegal. B. Stream has no where.

Answer. B. select returned a Stream.

4. Storing a query and adding to it

You store the builder. Intermediate calls return a new GlideQuery, so a variable can be extended. The graphic that says a terminal returns nothing GlideQuery-shaped is only about that last call. Calls before it are how the variable stays usable.

```
var open = new GlideQuery('incident').where('active', true);

// each where returns a new GlideQuery; open is unchanged
var p1 = open.where('priority', 1).count(); // number
var p2 = open.where('priority', 2).select('number'); // Stream

p2.where('priority', 1); // invalid: p2 is a Stream
```

Reassigning also works, because the right-hand side is still a GlideQuery: incidents = incidents.where('priority', 1).orderBy('number'). The terminal stays last. count and select are not stored if you still need to add conditions.

Knowledge check

What do you store if you still need to add a where? A. The Stream. B. The GlideQuery before the terminal.

Answer. B. A terminal does not return a GlideQuery.

5. Chain rules

Green methods return a GlideQuery. Amber methods execute and end the builder. Blue methods belong to Stream or Optional. Red rows are calls that fail because the receiver is the wrong object, or because the write is unbounded.

GlideQuery chain rules — validated

Every method is intermediate or terminal. Intermediate returns a new GlideQuery. Terminal executes and the chain is over. Scoped apps must use **global.GlideQuery**.

Official classification
Zurich / current docs
orderBy · deleteMultiple

● THE ONE RULE

`new GlideQuery` → `where` · `orderBy` · `limit` →

`select` / `count` / `update`

Builders first. A terminal returns a Stream, Optional, number, GlideRecord, or {rowCount}. Nothing GlideQuery-shaped comes back.

● INTERMEDIATE · KEEP CHAINING

Returns GlideQuery. Safe to chain.

● TERMINAL · CHAIN ENDS

Executes. Next call is Stream, Optional, or a value — not GlideQuery.

● INTERMEDIATE METHODS

`where` `orWhere` `whereNull` `whereNotNull` `orWhereNull` `orWhereNotNull` `orderBy`
`orderByDesc` `limit` `groupBy` `aggregate` `having` `withAcls` `disableWorkflow`
`forceUpdate` `disableAutoSysFields`

where(field, 'IN', [...]) is the membership form. No whereIn in the official list. limit() here runs before the query — use this, not Stream.limit().

● TERMINAL METHODS

`select` `selectOne` `get` `getBy` `count` `sum` `max` `min` `avg` `insert`
`insertOrUpdate` `update` `updateMultiple` `deleteMultiple` `toGlideRecord`

select → Stream (sys_id always included), selectOne / get / getBy / insert / update → Optional, count → number, updateMultiple → {rowCount}, deleteMultiple → nothing, parse() is static, neither.

● CHAINS THAT FAIL

INVALID	WHY
<code>select().where()</code>	select returned a Stream
<code>count().select()</code>	count returned a number
<code>select().orderBy()</code>	builders must come first
<code>insert().where()</code>	no row set to filter
<code>update()</code> without <code>sys_id where</code>	single-record update needs the primary key
<code>new GlideQuery().forEach()</code>	forEach is Stream-only
<code>get().get()</code>	second get unwraps an Optional
<code>instance.parse()</code>	parse is static: GlideQuery.parse(table, eq)

● AFTER THE TERMINAL

Not GlideQuery. Do not chain query builders onto these.

`Stream.forEach` `toArray` `map` `filter` `reduce` `find` `flatMap`
`limit`
`Optional.get` `orElse` `isPresent` `isEmpty` `ifPresent`

Optional.get() throws if empty. Prefer isPresent() when absence is normal. Stream.limit() after select works but reads then trims — mark it bad.

● FAIL-FAST VALIDATION · EXECUTES NOTHING

Unknown field
where('activated', '!=', true) → NiceError, not a full-table delete

Invalid choice
where('approval', 'donotexist') → lists allowed values

Wrong type
where('state', '8') on an integer field → type mismatch

insert / update also check field, type, and choice. Before a bulk write, run the same where() chain through count().

● NAMES, SETTLED

- **orderBy / orderByDesc** — official. orderByAsc is not in the API.
- **deleteMultiple()** — official. No single-record delete. del() is an alias. delete() is not documented.
- **update()** needs a primary-key where. Sets use updateMultiple().
- Bare updateMultiple / deleteMultiple with no where is unsupported. Docs require preceding where clauses; the exact error string is not quoted.

Sources: ServiceNow GlideQuery API (Zurich / current docs), GlideQuery blog (data integrity, no single delete), script-include alias del = deleteMultiple.

Figure 2. Official intermediate and terminal methods, invalid chains, and fail-fast checks.

Invalid chains

Call	Why it fails
<code>select().where()</code> / <code>select().orderBy()</code>	<code>select</code> returned a <code>Stream</code> . Builders must come first.
<code>count().select()</code>	<code>count</code> returned a number.
<code>insert().where()</code>	An insert has no row set to filter.
<code>update()</code> without a primary-key where	Single-record update requires the primary key, usually <code>sys_id</code> . Use <code>updateMultiple</code> for a set.
<code>new GlideQuery(table).forEach(fn)</code>	<code>forEach</code> is <code>Stream</code> -only. <code>select</code> first.
<code>get().get()</code>	The first <code>get</code> is <code>GlideQuery.get(sys_id)</code> . The second is <code>Optional.get</code> , and it throws if empty.
<code>instance.parse(...)</code>	<code>parse</code> is static: <code>GlideQuery.parse(table, encodedQuery)</code> .
<code>selectOne().forEach(fn)</code>	<code>selectOne</code> returned an <code>Optional</code> . <code>Optional</code> has no <code>forEach</code> .
<code>orElse(v).isPresent()</code>	<code>orElse</code> already unwrapped the <code>Optional</code> . The result is a plain value.



Knowledge check

Is `orderByAsc` an official method? A. Yes. B. No.

Answer. B. Official names are `orderBy` and `orderByDesc`.

6. When to use each object

Pick the object from the shape of the answer, not from habit. `GlideQuery` is the question. `Stream` is a list of answers. `Optional` is one answer that might be missing. `count`, `updateMultiple`, and `deleteMultiple` are neither: they return a number, a row count, or nothing.

Use	When	Why this one
<code>GlideQuery</code>	You are still deciding which rows. You may add a where later, or run the same filter as a count and as a select.	<code>where</code> and <code>orderBy</code> exist only here. Storing this is what lets you extend the query. A terminal throws that away.
<code>Stream</code> , via <code>select()</code>	The answer is a set: zero, one, or many rows, and you will loop, map, or collect them.	A set does not fit in an <code>Optional</code> . <code>select</code> is lazy, so map and filter cost nothing until <code>forEach</code> , <code>toArray</code> , or <code>reduce</code> .
<code>Optional</code> , via <code>selectOne()</code>	A filter should match one row, and no row is a normal result.	<code>selectOne</code> always returns an <code>Optional</code> , empty or not. You handle absence with <code>orElse</code> or <code>isPresent</code> instead of a null check on a <code>GlideRecord</code> .
<code>Optional</code> , via <code>get(sys_id)</code>	You already have the primary key.	<code>get</code> is the lookup. It still returns an <code>Optional</code> , because that <code>sys_id</code> may not exist.
<code>Optional</code> , via <code>getBy({...})</code>	You have field values, not a <code>sys_id</code> , and you only want one match.	<code>getBy</code> returns one row even if several match. Do not use it if you need every match — that is <code>select</code> .
<code>Optional.get()</code>	A missing row is a bug, not a case to handle.	<code>get</code> unwraps the <code>Optional</code> and throws if it is empty. That is the wrong call when absence is normal.

`Stream` versus `Optional` is the same choice as a list versus a single value. If you would write a `while (gr.next())` loop, use `select` and a `Stream`. If you would write `if (gr.get(sys_id))`, use `get` or `selectOne` and an `Optional`. Using `select` for one row forces a loop around a single object. Using `selectOne` for many rows silently keeps only the first.

Stream — a set of incidents

Use this when the caller needs every open P1, not one of them. limit goes on the GlideQuery so the database stops at 20. forEach is what reads the rows.

```
// why Stream: the result is a list, and zero rows is fine – forEach just does not run
var openP1 = new GlideQuery('incident')
  .where('active', true)
  .where('priority', 1)
  .orderBy('number')
  .limit(20);

openP1.select('number', 'short_description')
  .forEach(function (row) {
    gs.info(row.number + ' ' + row.short_description);
  });

// same builder, different terminal – a count, not a Stream
gs.info('open P1 count ' + openP1.count());
```

Optional — one user, who might not exist

Use this when the caller asked for one user by email. orElse covers the normal miss. get() is only there if a missing user should stop the script.

```
// why Optional: one row or none. orElse supplies the default without a throw
var user = new GlideQuery('sys_user')
  .where('email', 'abel.tuter@example.com')
  .selectOne('first_name', 'email')
  .orElse({ first_name: 'Nobody', email: '' });
gs.info(user.first_name);

// same lookup, absence is a bug – get throws on an empty Optional
var mustExist = new GlideQuery('sys_user')
  .get('5137153cc611227c000bbd1bd8cd2005', ['first_name'])
  .get();
gs.info(mustExist.first_name);
```

💡 Knowledge check

A list of incidents should use: A. select. B. selectOne.

Answer. A. A set is a Stream. selectOne keeps one row.

7. Every syntax that produces a Stream or an Optional

select() is the only GlideQuery method that creates a Stream. Nothing else on the builder does: not selectOne, not get, not count, not a where. Once you have that Stream, map, filter, limit, flatMap, and chunk return another Stream. They do not create the first one. find is the bridge out: Stream in, Optional out.

An Optional has more than one entrance. `selectOne`, `get`, and `getBy` wrap one row. `insert`, `update`, and `insertOrUpdate` wrap the written row. `sum`, `max`, `min`, and `avg` wrap one number, which is why `orElse(0)` is legal on them. `Stream.find` wraps the first matching row. `Optional.map` wraps whatever the function returns. `count` is not on this list: it returns a number. `updateMultiple` returns `{rowCount}`. `deleteMultiple` returns nothing.

Call	You are holding	Returns	Syntax
<code>select('f1', 'f2')</code>	GlideQuery	Stream	The only GlideQuery call that creates a Stream. <code>sys_id</code> is always included. No fields means every field.
<code>map / filter / limit</code>	Stream	Stream	Keep the Stream. Lazy until a Stream terminal. <code>limit</code> here trims after the query.
<code>flatMap(fn)</code>	Stream	Stream	<code>fn</code> must return a Stream, usually another <code>select()</code> . <code>flatMap</code> flattens it. <code>map</code> would nest Stream inside Stream.
<code>chunk(n)</code>	Stream	Stream	Stream of arrays of <code>n</code> items. Still a Stream, so <code>flatMap</code> or <code>forEach</code> can follow.
<code>forEach / reduce / toArray(n)</code>	Stream	nothing / value / array	These end the Stream and run the read. Nothing Stream-shaped comes back.
<code>find(fn)</code>	Stream	Optional	First match, or empty. Optional methods are legal after this, Stream methods are not.
<code>selectOne('f1')</code>	GlideQuery	Optional	One row matching the where, or empty. Several matches: you get one, not a Stream.
<code>get(sys_id, fields)</code>	GlideQuery	Optional	Primary-key lookup. Empty if that <code>sys_id</code> is missing.
<code>getBy({...}, fields)</code>	GlideQuery	Optional	One row by field values. One even if many match. Every match is <code>select</code> , not <code>getBy</code> .
<code>insert / update / insertOrUpdate</code>	GlideQuery	Optional	The written row, or empty if the write did not return one. <code>update</code> still needs a primary-key where.
<code>sum / max / min / avg</code>	GlideQuery	Optional	One number, or empty if nothing matched. <code>orElse(0)</code> is the usual unwrap.
<code>Optional.map(fn)</code>	Optional	Optional	Another Optional. Chain it before <code>orElse</code> or <code>get</code> . It does not make a Stream.
<code>isPresent / isEmpty</code>	Optional	boolean	The test. They do not unwrap.
<code>get() / orElse(v)</code>	Optional	the row or value	Unwrap. <code>get</code> throws if empty. <code>orElse</code> returns the default. Neither returns an Optional.

Paths that stay a Stream

```
// only select() creates the Stream
var numbers = new GlideQuery('incident')
  .where('active', true)
  .select('number') // Stream
  .filter(function (row) { return row.number; }) // Stream
  .map(function (row) { return row.number; }) // Stream
  .limit(10); // Stream – trims after the read
numbers.forEach(gs.info); // ends it

// flatMap: inner select() makes a Stream, flatMap keeps one Stream
new GlideQuery('sys_user')
  .whereNotNull('first_name')
  .select('first_name')
  .flatMap(function (user) {
```

```
return new GlideQuery('incident')
.where('caller_id', user.sys_id)
.select('number');
})
.forEach(function (inc) { gs.info(inc.number); });
```

A where after any of these is still invalid. The Stream was created by select, and where is not a Stream method. Put the filter on the GlideQuery, or use filter on the Stream if you mean an in-script check.

Paths that produce an Optional

```
// one row from a filter
var byEmail = new GlideQuery('sys_user')
.where('email', 'abel.tuter@example.com')
.selectOne('first_name')
.orElse({ first_name: 'Nobody' });

// one row from a primary key
var byId = new GlideQuery('sys_user')
.get('5137153cc611227c000bbd1bd8cd2005', ['first_name'])
.orElse(null);

// one number – sum returns an Optional, not a Stream and not a raw number
var faults = new GlideQuery('cmdb_ci').sum('fault_count').orElse(0);

// Stream.find is the other entrance: select made the Stream, find makes the Optional
new GlideQuery('incident')
.where('active', true)
.select('number', 'short_description')
.find(function (row) { return row.short_description; })
.ifPresent(function (row) { gs.info(row.number); });

// written row
var created = new GlideQuery('incident')
.insert({ short_description: 'opened from script' })
.orElse(null);
```

selectOne on a set does not give you a Stream of the rest. It gives you one Optional. getBy does the same. If you need every match, the call is select, and the object is a Stream.

Knowledge check

Which GlideQuery call creates a Stream? A. select. B. selectOne. C. sum.

Answer. A. selectOne and sum return an Optional.

8. Stream versus Optional syntax

Use `select` when you expect many rows, including zero. Use `selectOne`, `get`, or `getBy` when you expect one row that might be missing. `get(sys_id)` looks up the primary key. `getBy({field: value})` returns one match even if several rows qualify. `insert`, `update`, and `insertOrUpdate` also return an `Optional` of the written row.

SERVICENOW · AFTER THE TERMINAL

Stream vs Optional — valid and invalid

`select()` returns a `Stream` (many rows). `selectOne()`, `get()`, and `getBy()` return an `Optional` (one row, or none). Neither has `where()`.

STREAM — FROM SELECT()

Still a `Stream`, so you can chain them:

`map` `filter` `limit` `flatMap`

These end the `Stream`: `forEach` `reduce` `toArray` `find` — `find` returns an `Optional`.

```
new GlideQuery('incident')
  .where('active', true)
  .select('number')
  .filter(function (row) { return row.number; })
  .map(function (row) { return row.number; })
  .forEach(gs.info);
```

```
var rows = new GlideQuery('incident')
  .select('number').toArray(50);
```

```
.select('number').where('active', true);
.select('number').orderBy('number');
.forEach(gs.info).map(fn);
new GlideQuery('incident').filter(fn);
```

`map/filter/limit` are lazy. Rows are read when `forEach`, `reduce`, `toArray`, or `find` runs. `Stream.limit` trims after the query. `GlideQuery.limit`, before `select`, is the one that belongs in the query.

OPTIONAL — FROM SELECTONE / GET / GETBY

`isPresent` `isEmpty` return a boolean. `get` returns the row or throws. `orElse` returns the row or a default. `ifPresent` runs only if a row exists. `map` returns another `Optional`.

```
var user = new GlideQuery('sys_user')
  .where('email', 'abel.tuter@example.com')
  .selectOne('first_name')
  .orElse({ first_name: 'Nobody' });

var opt = new GlideQuery('sys_user')
  .selectOne('first_name');
if (opt.isPresent()) gs.info(opt.get().first_name);

.selectOne('first_name').where('active', true);
.selectOne('first_name').forEach(gs.info);
.selectOne('first_name').get(); // throws if empty
.orElse({ first_name: 'Nobody' }).isPresent();
```

`orElse` already unwrapped the `Optional`, so the result is a plain object. `insert`, `update`, and `insertOrUpdate` also return an `Optional`. `Stream.find` is the bridge: `Stream` in, `Optional` out.

Figure 3. Valid and invalid syntax once the builder has ended.

Stream

`map`, `filter`, `limit`, and `flatMap` return a `Stream`, so they chain, and they are lazy. `forEach`, `reduce`, and `toArray` end the `Stream` and run the query. `find` ends the `Stream` and returns an `Optional`, so `Optional` methods are legal after it. `toArray(n)` needs a cap. Prefer `GlideQuery.limit` before `select`. `Stream.limit` works, then trims, which reads more than you keep.

```
new GlideQuery('incident')
  .where('active', true)
  .select('number', 'short_description')
  .filter(function (row) { return row.short_description; })
  .map(function (row) { return row.number; })
  .forEach(gs.info);
```

```
var rows = new GlideQuery('incident').where('active', true).select('number').toArray(50);
```

```
new GlideQuery('task').select('description')
  .find(function (row) { return row.description; })
  .ifPresent(function (row) { gs.info(row.sys_id); });
```

Optional

isPresent and isEmpty return a boolean. get returns the row or throws "get() called on empty Optional" — use it only when a missing row is a bug. orElse(default) returns the row or the default, already unwrapped. ifPresent(fn) runs only when a row exists. map(fn) returns another Optional, so it can sit before orElse or get.

```
var user = new GlideQuery('sys_user')
    .where('email', 'abel.tuter@example.com')
    .selectOne('first_name', 'email')
    .orElse({ first_name: 'Nobody', email: '' });

var opt = new GlideQuery('sys_user').where('last_name', 'Luddy').selectOne('first_name');
if (opt.isPresent()) {
    gs.info(opt.get().first_name);
}
```

💡 Knowledge check

orElse().isPresent() is: A. Valid. B. Invalid.

Answer. B. orElse already unwrapped the Optional.

9. Writes

update needs a where on the primary key and returns an Optional of the updated row. updateMultiple takes a changes object and returns {rowCount}. There is no single-record delete. deleteMultiple deletes the rows matched by the preceding where clauses; del is an alias of deleteMultiple, not a second API. Docs describe both bulk writes as operating on the preceding where clauses. A bare updateMultiple or deleteMultiple with no where is unsupported. The exact error string is not quoted in the docs, so pair the same where chain with count before either call. Do not use deleteMultiple on tables with currency fields; currency child rows can be orphaned. Delete those one by one.

```
var n = new GlideQuery('incident').where('active', true).where('priority', 1).count();
gs.info('would update ' + n);

new GlideQuery('incident')
    .where('sys_id', '0123456789abcdef0123456789abcdef')
    .update({ short_description: 'corrected' });

new GlideQuery('incident')
    .where('active', false)
    .where('priority', 5)
    .updateMultiple({ urgency: 3 });
```

💡 Knowledge check

Which updates one row? A. updateMultiple with no where. B. update where sys_id.

Answer. B. update requires the primary key.

10. Names that cheat sheets disagree on

Use this	Not this	Note
orderBy / orderByDesc	orderByAsc	orderByAsc appears on some unofficial sheets. It is not in the official method list.
deleteMultiple	delete() / Del()	No single-record delete. del is the script-include alias of deleteMultiple.
where(field, 'IN', [...])	whereIn / whereNotIn	IN is an operator on where, not a separate documented method.
global.GlideQuery	GlideQuery in a scoped app	Scoped scripts must prefix the global scope.
GlideQuery.parse(table, eq)	query.parse(...)	parse is static.

🔦 Knowledge check

Documented delete is: A. delete(). B. deleteMultiple().

Answer. B. del is only an alias of deleteMultiple.

11. Progressive examples

Each branch has five steps and a wildcard. Easy examples say why the object is that object. Color: **green** is a GlideQuery builder call. **amber** is a GlideQuery terminal — the builder ends here. **blue** is a Stream call. **violet** is an Optional call. A terminal is amber even when what it returns is a Stream or an Optional.

GlideQuery — the builder

Easy. One filter, then the terminal. where is green because you could still add another. select is amber because the next call is no longer a GlideQuery.

```
new GlideQuery('incident')
  .where('active', true)
  .select('number')
  .forEach(function (row) { gs.info(row.number); });
```

Easy. Same builder, two terminals. count is amber and returns a number, so it is not a Stream and not an Optional.

```
var open = new GlideQuery('incident').where('active', true);
var n = open.where('priority', 1).count();
var rows = open.where('priority', 2).select('number');
```

Medium. One user. selectOne is the terminal. orElse is violet because you are already on an Optional.

```
var user = new GlideQuery('sys_user')
  .where('email', 'abel.tuter@example.com')
  .selectOne('first_name')
  .orElse({ first_name: 'Nobody' });
```

Medium. A set write. updateMultiple is amber. It returns {rowCount}, not a Stream. Count first so you know the size of the set.

```
var q = new GlideQuery('incident')
  .where('active', false)
  .where('priority', 5);
gs.info(q.count());
q.updateMultiple({ urgency: 3 });
```

Hard. Grouped aggregate. aggregate, groupBy, and having are still the builder. select() with no fields is the terminal and returns a Stream of group rows.

```
new GlideQuery('task')
  .where('active', true)
  .aggregate('count')
  .groupBy('priority')
  .having('count', 'priority', '>', 4)
  .select()
  .forEach(function (row) { gs.info(row.priority); });
```

Wildcard. An encoded query is not a where chain. parse is static, then select is still the terminal that creates the Stream.

```
GlideQuery.parse('incident', 'active=true^priority=1')
  .limit(10)
  .select('number')
  .toArray(10);
```

Stream — only after select

Easy. forEach is blue. It is what reads the rows. Zero rows means the function never runs, which is why a list uses a Stream.

```
new GlideQuery('incident')
  .where('active', true)
  .limit(20)
  .select('number', 'short_description')
  .forEach(function (row) {
    gs.info(row.number + ' ' + row.short_description);
  });
```

Easy. toArray(n) is blue and ends the Stream. The cap is required. You now hold an array, so forEach after this is the array method, not the Stream method.

```
var rows = new GlideQuery('incident')
  .where('active', true)
  .select('number')
  .toArray(50);
```

Medium. filter and map are blue and still a Stream. They are lazy. The read waits for forEach.

```
new GlideQuery('incident')
  .where('active', true)
  .select('number', 'short_description')
  .filter(function (row) { return row.short_description; })
```

```
.map(function (row) { return row.number; })
.forEach(gs.info);
```

Medium. `find` is blue and ends the Stream. `ifPresent` is violet: `find` returned an `Optional`.

```
new GlideQuery('incident')
.where('active', true)
.select('number', 'short_description')
.find(function (row) { return row.short_description; })
.ifPresent(function (row) { gs.info(row.number); });
```

Hard. One-to-many. Caller to that caller's incidents. The inner `select` creates a Stream; `flatMap` keeps a single Stream instead of a Stream of Streams.

```
new GlideQuery('sys_user')
.where('active', true)
.limit(10)
.select('first_name')
.flatMap(function (user) {
return new GlideQuery('incident')
.where('caller_id', user.sys_id)
.where('active', true)
.select('number');
})
.forEach(function (inc) { gs.info(inc.number); });
```

Wildcard. `reduce` is blue and ends the Stream with one value. Use `sum` on the `GlideQuery` when the database can do it. `reduce` is for a shape `sum` cannot express.

```
var score = new GlideQuery('incident')
.where('active', true)
.select('impact', 'urgency')
.reduce(function (acc, row) {
return acc + (row.impact * row.urgency);
}, 0);
```

Optional — one value, or none

Easy. No row is normal. `orElse` is violet and unwraps, so the variable is a plain object. That is why `isPresent` after `orElse` is invalid.

```
var user = new GlideQuery('sys_user')
.where('email', 'abel.tuter@example.com')
.selectOne('first_name', 'email')
.orElse({ first_name: 'Nobody', email: '' });
```

Easy. `isPresent` is violet and returns a boolean. `get` inside the `if` is the `unwrap`. `get` on an empty `Optional` throws, so the check comes first.

```
var opt = new GlideQuery('sys_user')
.where('last_name', 'Luddy')
```

```
.selectOne('first_name');
if (opt.isPresent()) gs.info(opt.get().first_name);
```

Medium. You already have the key. get is amber: it is the GlideQuery terminal. orElse is violet.

```
var user = new GlideQuery('sys_user')
.get('5137153cc611227c000bbd1bd8cd2005', ['first_name'])
.orElse(null);
```

Medium. A sum is one number, so it is an Optional, not a Stream. orElse(0) covers no matching rows.

```
var faults = new GlideQuery('cmdb_ci')
.where('install_status', 1)
.sum('fault_count')
.orElse(0);
```

Hard. One-to-many, but you only want the parent and one field off the caller. Dot-walk in selectOne. Still an Optional, because the incident is one row.

```
var inc = new GlideQuery('incident')
.where('number', 'INC0010001')
.selectOne('number', 'caller_id', 'caller_id.email')
.orElse(null);
```

Wildcard. Many-to-many, stopped at the first match. Group to member rows is a Stream. find turns it into an Optional.

```
new GlideQuery('sys_user_grmember')
.where('group.name', 'CAB Approval')
.select('user', 'user.email')
.find(function (m) { return m.user; })
.ifPresent(function (m) { gs.info(m['user.email']); });
```

Knowledge check

Amber in the examples means: A. A Stream call. B. A GlideQuery terminal.

Answer. B. Blue is Stream. Violet is Optional.

12. Beside GlideRecord and GlideAggregate

Use GlideRecord when you already hold a record and need setValue, setWorkflow, or a method GlideQuery does not wrap. Use GlideAggregate when you are already in that style. The GlideQuery form is shorter and fails on a bad field. toGlideRecord is the escape hatch: it is a terminal, and you still call query() on the GlideRecord it returns.

Query a set. GlideRecord ignores a bad field name. GlideQuery throws before the read.

```
// GlideRecord
var gr = new GlideRecord('incident');
```

```

gr.addQuery('active', true);
gr.orderBy('number');
gr.setLimit(20);
gr.query();
while (gr.next()) gs.info(gr.getValue('number'));

// GlideQuery – select is the terminal, forEach is the Stream read
new GlideQuery('incident')
  .where('active', true)
  .orderBy('number')
  .limit(20)
  .select('number')
  .forEach(function (row) { gs.info(row.number); });

```

One row by sys_id. GlideRecord.get returns false if missing. GlideQuery.get returns an Optional, so the miss is orElse, not a boolean.

```

var gr = new GlideRecord('sys_user');
if (gr.get('5137153cc611227c000bbd1bd8cd2005')) gs.info(gr.getValue('first_name'));

var user = new GlideQuery('sys_user')
  .get('5137153cc611227c000bbd1bd8cd2005', ['first_name'])
  .orElse(null);

```

Count. GlideAggregate needs addAggregate, query, next, getAggregate. count is the GlideQuery terminal and returns a number.

```

var ga = new GlideAggregate('incident');
ga.addQuery('active', true);
ga.addAggregate('COUNT');
ga.query();
ga.next();
var n = parseInt(ga.getAggregate('COUNT'), 10);

var n2 = new GlideQuery('incident').where('active', true).count();

```

Group and sum. having is a builder call. select is still required: the groups come back as a Stream.

```

var ga = new GlideAggregate('incident');
ga.addQuery('active', true);
ga.addAggregate('SUM', 'reassignment_count');
ga.groupBy('priority');
ga.query();
while (ga.next()) gs.info(ga.getAggregate('SUM', 'reassignment_count'));

new GlideQuery('incident')
  .where('active', true)
  .aggregate('sum', 'reassignment_count')

```

```

.groupBy('priority')
.select()
.forEach(function (row) { gs.info(row.priority); });

```

🔔 Knowledge check

A bad field name on GlideRecord: A. Throws. B. Is ignored.

Answer. B. The query can match every row. GlideQuery throws.

13. One-to-many and many-to-many

GlideQuery has no addJoinQuery. Dot-walk in where or select for a parent field. flatMap is the child query: one-to-many, or the two hops of a many-to-many. Use a Stream when you want every child. Use an Optional when you want the parent, or the first child only.

One-to-many, Stream, shape only. flatMap runs a query per parent. That is an N+1 trap. Section 21 shows the IN batch. Inner select creates the child Stream. flatMap flattens.

```

new GlideQuery('sys_user')
.where('active', true)
.limit(5)
.select('sys_id')
.flatMap(function (user) {
return new GlideQuery('incident')
.where('caller_id', user.sys_id)
.where('active', true)
.select('number', 'caller_id');
})
.toArray(100);

```

One-to-many, Optional. One incident, with the caller's email dot-walked onto that single row. selectOne, not select, because you asked for one incident.

```

var inc = new GlideQuery('incident')
.get('0123456789abcdef0123456789abcdef', ['number', 'caller_id.email'])
.orElse(null);

```

Many-to-many, Stream. Group to sys_user_grmember to user, then to that user's active tasks. Two flatMaps, one Stream at the end.

```

new GlideQuery('sys_user_group')
.where('name', 'CAB Approval')
.select('sys_id')
.flatMap(function (group) {
return new GlideQuery('sys_user_grmember')
.where('group', group.sys_id)
.select('user');
})

```

```

.flatMap(function (member) {
return new GlideQuery('task')
.where('assigned_to', member.user)
.where('active', true)
.select('number');
})
.forEach(function (task) { gs.info(task.number); });

```

Many-to-many, Optional. Same membership table, stopped at the first member. find is the Stream terminal that returns an Optional.

```

var firstMember = new GlideQuery('sys_user_grmember')
.where('group.name', 'CAB Approval')
.select('user', 'user.name')
.find(function (m) { return m.user; })
.orElse(null);

```

Dot-walk field names depend on the instance. If caller_id.email or user.name is not in the select list on your release, select the reference sys_id and load the parent with get, which returns an Optional.

Knowledge check

Every child row uses: A. find. B. flatMap.

Answer. B. find returns an Optional of the first match.

14. Grouped sum — the object, not a method

There is no group.sum() method. aggregate('sum', field) plus groupBy plus select() returns a Stream of objects. Each object has a group block and a sum block. map can flatten that. sum() with no groupBy is the other tool: one number, wrapped in an Optional.

```

// Stream of group objects. select is the terminal.
new GlideQuery('task')
.where('active', true)
.groupBy('priority')
.aggregate('sum', 'reassignment_count')
.select()
.forEach(function (row) {
// row.group.priority
// row.sum.reassignment_count
gs.info(row.group.priority + ' ' + row.sum.reassignment_count);
});

// same Stream, map flattens the blocks, forEach still ends it
new GlideQuery('task')
.where('active', true)

```

```

.groupBy('priority')
.aggregate('sum', 'reassignment_count')
.select()
.map(function (row) {
return { priority: row.group.priority, total: row.sum.reassignment_count };
})
.forEach(gs.info);

// no group – one number, Optional, not a Stream
var total = new GlideQuery('task')
.where('active', true)
.sum('reassignment_count')
.orElse(0);

```

A grouped row looks like { group: { priority: 1 }, sum: { reassignment_count: 11 } }. count grouped the same way uses a count block. having stays on the builder, before select.

Knowledge check

row.sum is: A. A method. B. A block on the grouped row.

Answer. B. There is no group.sum() method.

15. Quiz — easy

Decide the object before you write the call. Answers are under each question.

1. You need every open incident number. Which terminal?

A. select, then a Stream call. B. selectOne, then orElse.

Answer. A. A set is a Stream. select is the only GlideQuery call that creates one. selectOne keeps a single row.

2. True or false: a missing user should use get() on the Optional so the script gets null.

Answer. False. Optional.get throws if empty. A normal miss is orElse or isPresent. get is for when missing is a bug.

3. Which line is invalid?

```

new GlideQuery('incident')
.where('active', true)
.select('number')
.where('priority', 1);

```

Answer. The third call. select returned a Stream. where is a builder method. Put where before select, or use filter on the Stream.

4. Which block logs one user, with a default if the email is missing?

A. where email, select('first_name'), forEach. B. where email, selectOne('first_name'), orElse({ first_name: 'Nobody' }).

Answer. B. One row or none is an Optional. forEach is a Stream call and does not supply a default.

🔦 Knowledge check

A missing row and a default should use: A. orElse. B. get.

Answer. A. get throws if empty.

16. Quiz — medium

5. Would this work? You want the open P1 count, and you may add another where later.

```
var open = new GlideQuery('incident')
  .where('active', true)
  .count();
open = open.where('priority', 1);
```

Answer. No. count is a terminal and returns a number. Store the builder, then call count on the extended builder.

```
var open = new GlideQuery('incident').where('active', true);
var n = open.where('priority', 1).count();
```

6. Which line is invalid?

```
new GlideQuery('sys_user')
  .selectOne('first_name')
  .orElse({ first_name: 'Nobody' })
  .isPresent();
```

Answer. isPresent. orElse already unwrapped the Optional. The value is a plain object.

7. You want the first 20 open incidents as an array. Which is the right set?

A. where active, limit(20), select('number'), toArray(20).

B. where active, select('number'), where priority, toArray(20).

C. where active, selectOne('number'), toArray(20).

D. where active, count(), toArray(20).

Answer. A. limit belongs on the builder. toArray is the Stream terminal. B calls where on a Stream. C is an Optional. D is a number.

🔦 Knowledge check

Can you where() the value returned by count()? A. Yes. B. No.

Answer. B. count returned a number.

17. Quiz — hard

8. Would this log the reassignment total per priority?

```
new GlideQuery('task')
  .groupBy('priority')
  .aggregate('sum', 'reassignment_count')
  .select()
  .forEach(function (row) {
    gs.info(row.group.priority + ' ' + row.sum.reassignment_count);
  });
```

Answer. Yes. select returns a Stream of { group, sum } objects. row.sum is a block on that object, not a method. map can flatten it. sum() with no groupBy is one Optional number, not these rows.

9. Which line is invalid if you need every task for every member of CAB Approval?

```
new GlideQuery('sys_user_grmember')
  .where('group.name', 'CAB Approval')
  .select('user')
  .find(function (m) { return m.user; })
  .flatMap(function (m) {
    return new GlideQuery('task').where('assigned_to', m.user).select('number');
  });
```

Answer. flatMap after find. find returned an Optional. flatMap is a Stream call. find is also the wrong tool: it keeps the first member. Use flatMap on the membership Stream.

10. Active callers, every open incident number, capped at 100. Which set?

- A. select users, flatMap a child select of incidents, toArray(100).
- B. selectOne user, flatMap incidents, forEach.
- C. select users, forEach, and inside it select incidents, where on that Stream.
- D. groupBy caller_id, sum(number), orElse(0).

Answer. A. One-to-many of every child is a Stream plus flatMap. The inner select creates the child Stream. B is one user. C calls where on a Stream. D is an aggregate of a string field, and orElse is the ungrouped Optional path.

18. Performance

These are the costs this note left out until now.

limit on the GlideQuery is part of the query. Stream.limit runs after select and trims in script. The database still did the wider read.

`select('number')` reads those columns. `select()` with no fields reads the row. On task and cmdb_ci that is a wide row. Project the fields you use.

`flatMap` is one query per parent row. A user Stream of 10 with an incident select inside is 1 + 10 queries. A many-to-many with two `flatMap`s multiplies again. If the set is large, an encoded query or a `GlideAggregate` is the better tool.

`sum`, `count`, `max`, `min`, and `avg` run in the database. `Stream.reduce` loads the rows and folds them in script. Use `reduce` only when the database aggregate cannot express the total.

`count()` then `select()` is two queries. Fine as a guard before `updateMultiple`. Wasteful if you only needed the rows.

`toArray(n)` materializes n rows. `forEach` can stop early in script, but the query cap is still `GlideQuery.limit`, not a break inside `forEach`.

`deleteMultiple` is one operation and skips per-row script. Do not use it on tables with currency fields: `fx_currency` rows can be orphaned. It also does not delete attachments. `withAcls()` adds ACL checks and costs more than the default.

Dot-walk in where or select adds a join. `groupBy` plus aggregate is one grouped query. Doing the same with `flatMap` and a child count is N queries.

🔔 Knowledge check

`Stream.limit(10)` after select is the same cost as `GlideQuery.limit(10)` before select. True or false?

Answer. False. The builder limit is in the query. The Stream limit trims after the read.

19. Critique of this note

Performance was missing. Section 18 is the correction. The examples never said that `flatMap` multiplies queries, or that `select()` with no fields reads the whole row.

Answers sit under the questions, so this is a self-check, not a closed quiz. A reader can see the answer without trying.

having in one unofficial sheet uses a different argument order from the product docs. This note follows the docs: `having('sum', 'reassignment_count', '>', 4)`. Confirm on your release.

Dot-walk field names and the group / sum blocks are from the Zurich reference. An older instance may not project `caller_id.email` the same way. The escape is get on the reference `sys_id`.

Domain separation, `withAcls`, and business-rule cost on update versus `updateMultiple` are named, not measured. No timing numbers are claimed.

The free Zurich course does not teach `GlideQuery`. The instructor-led SSNF lab does. Linking the free course without that distinction would send a reader to `GlideRecord` only.

🔔 Knowledge check

Which cost does this note still not measure?

Answer. ACL and business-rule time. Section 18 says they exist. It does not give numbers.

20. Courses and API references

Australia uses the same Now Learning catalog. No AU edition of SSNF or of the on-demand course was found. Book the virtual instructor-led session from Australia.

[Scripting in ServiceNow Fundamentals \(SSNF\)](#) — instructor-led, 3 days, syllabus KB0012764. Day 2 lab: Explore GlideQuery.

[SSNF course page](#) — Now Learning. Prerequisites include Welcome to ServiceNow and JavaScript.

[Scripting in ServiceNow](#) — free on-demand, Zurich release. Server-side scripting module: GlideRecord, GlideSystem, GlideDateTime. Not a GlideQuery course.

[GlideQuery](#) — Zurich developer reference. Intermediate and terminal lists, select, selectOne, aggregate, get, update.

[Stream](#) — forEach, map, filter, limit, flatMap, find, reduce, toArray, chunk.

[Optional](#) — isPresent, isEmpty, get, orElse, ifPresent, map.

[GlideQuery product docs](#) — scoped and global, including deleteMultiple and orderBy.

[GlideRecord · GlideAggregate](#) — the side-by-side alternatives.

[GlideQuery blog, part 1](#) — official series. Part 2 is writes. Parts 6 and 7 are Stream.

🔔 Knowledge check

Which official course has an Explore GlideQuery lab?

Answer. [Scripting in ServiceNow Fundamentals \(SSNF\)](#), not the free Zurich on-demand course.

21. Corrections: ACLs, operators, display values, and the N+1 trap

A review flagged case errors and broken quotes. Those are not in this PDF. selectOne and ifPresent are spelled that way. where(field, 'IN', [...]) is shorthand in a names table, not a snippet to paste. The gaps below were real.

N+1. The flatMap examples show the object shape: a child select returns a Stream, flatMap keeps one Stream. They are the wrong production query for a large set. Each parent row fires another query. Batch the keys and use IN.

```
// production: one child query, not one per user
var ids = new GlideQuery('sys_user')
  .where('active', true)
  .limit(10)
  .select('sys_id')
  .map(function (user) { return user.sys_id; })
  .toArray(10);
new GlideQuery('incident')
  .where('caller_id', 'IN', ids)
  .where('active', true)
  .select('number', 'caller_id')
  .forEach(function (row) { gs.info(row.number); });
```

Memory. `toArray(n)` and `Stream.limit` build JavaScript objects for the rows already read. `GlideQuery.limit` is the cap that belongs in the query. A large `toArray` can exhaust the script transaction. `chunk(n)` walks a `Stream` in batches. It does not remove the N+1 cost inside a `flatMap`.

```
new GlideQuery('incident')
  .where('active', true)
  .limit(200)
  .select('number')
  .chunk(50)
  .forEach(function (batch) { gs.info(batch.length); });
```

ACLs. `GlideQuery` does not honor read and write ACLs unless you call `withAcls()`. The docs say that method runs the query through `GlideRecordSecure`. An endpoint or widget server script that forgets it can return rows the user must not see. Scoped `GlideRecord` is the one that enforces ACLs by default.

```
new global.GlideQuery('sys_user')
  .withAcls()
  .limit(5)
  .select('first_name')
  .forEach(function (user) { gs.info(user.first_name); });
```

Display values. `select` returns the stored value. A reference comes back as a `sys_id`. Append `$DISPLAY` to the field name to get the display value. The key in the row is `caller_id$DISPLAY`.

```
new GlideQuery('incident')
  .where('active', true)
  .limit(5)
  .select('number', 'caller_id$DISPLAY')
  .forEach(function (row) { gs.info(row.number + ' ' + row['caller_id$DISPLAY']); });
```

Operators. The second argument is the operator. Equality is the two-argument form. Comparison, `STARTSWITH`, `CONTAINS`, `NOT IN`, and `BETWEEN` use three arguments. `BETWEEN` takes a two-item array on the releases that accept it. Confirm that one on your instance.

```
new GlideQuery('incident')
  .where('priority', '<', 3)
  .where('short_description', 'STARTSWITH', 'Email')
  .where('state', 'NOT IN', [7, 8])
  .select('number');
```

AND and OR. `orWhere` after `where` is a flat OR, and order matters. The docs say not to put `where` after `orWhere`. Grouped precedence is a nested `GlideQuery` passed to `where`: active AND (priority 1 OR severity 1).

```
new GlideQuery('incident')
  .where('active', true)
  .where(new GlideQuery()
    .where('priority', 1)
    .orWhere('severity', 1))
  .limit(5)
  .select('number');
```

Workflow. `disableWorkflow()` is the builder form of `setWorkflow(false)`. The docs say it skips business rules, flows, workflows, and audit for that write. `disableAutoSysFields()` stops the `sys_updated_on` bump. `forceUpdate()` writes even when no field changed. These are how you migrate a GlideRecord write that turned engines off.

```
new GlideQuery('incident')
  .disableWorkflow()
  .where('active', true)
  .where('priority', 5)
  .updateMultiple({ urgency: 3 });
```

NiceError. A bad field, choice, or type throws before the write. Catch it if a production script must log and continue. Do not catch it to ignore a typo on a delete.

```
try {
  new GlideQuery('incident')
    .where('activated', true)
    .select('number')
    .forEach(gs.info);
} catch (e) {
  gs.error(e.message);
}
```

`parse(table, encodedQuery)` is the static way to reuse an encoded query. It does not re-check every operator the way a where chain does. Prefer where when you want the fail-fast field check. GlideQuery also allocates a new object per chained call. For a fix script in the tens of thousands of rows, a plain GlideRecord loop can be the lighter tool.

Knowledge check

A widget server script that must hide rows the user cannot read needs which call? A. `select` only. B. `withAcls()` before `select`.

Answer. B. GlideQuery does not honor ACLs unless `withAcls()` is called.

22. Terms, a sample set, and output quizzes

Builder means the GlideQuery object returned by `new GlideQuery` and by `where`, `orderBy`, `limit`, `groupBy`, and `aggregate`. You can store it and add another where. Terminal means the call that does not return a builder: `select` returns a Stream, `selectOne` and `sum` return an Optional, `count` returns a number, `updateMultiple` returns {rowCount}. Stream calls are `forEach`, `map`, `filter`, `flatMap`, `toArray`. Optional calls are `orElse`, `isPresent`, `ifPresent`, and the unwrapping `get`. `current` and `previous` are GlideRecord. `gs` is GlideSystem. None of those is a builder.

Sample rows. Five incidents. `caller_id` is stored as a `sys_id`. The display name is what `caller_id$DISPLAY` would return.

number	caller_id	display	priority	sys_mod_count	active
INC001	aaa	Abel	1	2	true
INC002	aaa	Abel	1	4	true
INC003	bbb	Beth	2	1	true

number	caller_id	display	priority	sys_mod_count	active
INC004	bbb	Beth	1	3	true
INC005	ccc	Chen	3	5	false

A list is select, then a Stream. Projected output is two lines, Abel then Beth, because Chen is inactive and limit is 2 after orderBy number.

```
new GlideQuery('incident')
  .where('active', true)
  .orderBy('number')
  .limit(2)
  .select('number', 'caller_id$DISPLAY')
  .forEach(function (row) {
gs.info(row.number + ' ' + row['caller_id$DISPLAY']);
});
// INC001 Abel
// INC002 Abel
```

One row is an Optional. Projected output is the string Beth, not a list.

```
var name = new GlideQuery('incident')
  .where('number', 'INC003')
  .selectOne('caller_id$DISPLAY')
  .map(function (row) { return row['caller_id$DISPLAY']; })
  .orElse('missing');
// Beth
```

Grouped aggregates. There is no group.sum() method. aggregate plus groupBy plus select returns a Stream of objects. Each object has a group block and one block per aggregate. Two aggregates on the active rows, grouped by caller, project to Abel 2 and 6, Beth 2 and 4. Chen is excluded by where active.

```
new GlideQuery('incident')
  .where('active', true)
  .groupBy('caller_id')
  .aggregate('count')
  .aggregate('sum', 'sys_mod_count')
  .select()
  .forEach(function (row) {
gs.info(row.group.caller_id + ' ' + row.count + ' ' + row.sum.sys_mod_count);
});
// aaa 2 6
// bbb 2 4
```

The ungrouped sum is one Optional, not those rows. Active sys_mod_count totals 2+4+1+3 = 10.

```
var total = new GlideQuery('incident')
  .where('active', true)
  .sum('sys_mod_count')
```

```
.orElse(0);  
// 10
```

Output quiz. Using the five rows above, what does this log?

```
new GlideQuery('incident')  
.where('priority', 1)  
.where('active', true)  
.select('number')  
.map(function (row) { return row.number; })  
.forEach(gs.info);
```

Answer. INC001 and INC002 and INC004. INC003 is priority 2. INC005 is priority 3 and inactive. map still returns a Stream, so forEach logs numbers, not row objects.

Second output quiz. What is the value of n?

```
var n = new GlideQuery('incident')  
.where('caller_id', 'bbb')  
.count();
```

Answer. 2. count is a terminal and returns a number. Both Beth rows match. It is not a Stream, so forEach on n would fail.

Appendix. Not GlideQuery

These are the APIs the opening table names. Sample output assumes the five incidents above and a user Abel whose sys_id is aaa.

GlideRecord. current in a business rule is this object. get returns true or false, not an Optional. A bad field name is ignored.

```
var gr = new GlideRecord('incident');  
gr.addQuery('number', 'INC001');  
gr.query();  
if (gr.next()) gs.info(gr.getValue('number') + ' ' + gr.getDisplayValue('caller_id'));  
// INC001 Abel  
  
if (current.priority == 1) current.setAbortAction(true); // business rule only
```

GlideAggregate. The same grouped sum, in the API you use for MONTH() and for a query you already wrote this way. getAggregate returns a string.

```
var ga = new GlideAggregate('incident');  
ga.addQuery('active', true);  
ga.addAggregate('COUNT');  
ga.addAggregate('SUM', 'sys_mod_count');  
ga.groupBy('caller_id');  
ga.query();  
while (ga.next()) {
```

```
gs.info(ga.getValue('caller_id') + ' ' + ga.getAggregate('COUNT') + ' ' + ga.getAggregate('SUM', 'sys_mod_count'));
}
// aaa 2 6
// bbb 2 4
```

gs is GlideSystem. It does not query a table. It logs, reads the session, and queues events.

```
gs.info('open by Abel');
gs.getUserID(); // sys_id of the session user
gs.getUser().getName(); // display name
gs.eventQueue('incident.updated', current, gs.getUserID(), gs.getUserName());
```

GlideAjax is the browser path. The client never constructs a GlideQuery. The script include does, and returns a string or JSON.

```
// client script
var ga = new GlideAjax('IncidentQueryAjax');
ga.addParam('sysparm_name', 'callerOf');
ga.addParam('sysparm_number', 'INC001');
ga.getXMLAnswer(function (answer) { g_form.addInfoMessage(answer); });

// script include, server
callerOf: function () {
var row = new global.GlideQuery('incident')
.where('number', this.getParameter('sysparm_number'))
.selectOne('caller_id$DISPLAY')
.orElse({ 'caller_id$DISPLAY': 'missing' });
return row['caller_id$DISPLAY']; // Abel
}
```

GlideSysAttachment and gs.eventQueue are not queries. deleteRecord is the one-row delete GlideQuery does not have. addJoinQuery is the join GlideQuery does not have. flatMap is the shape demo. where IN is the batch.

Knowledge check

On the five sample rows, active sum of sys_mod_count is: A. 15. B. 10. C. a Stream.

Answer. B. 10. Chen is inactive. sum returns an Optional, and orElse unwraps it to a number.

24. A simple count: number, Stream, Optional, GlideRecord

Same question, four shapes. On the five sample rows, how many incidents are active? INC001 through INC004 are active. INC005 is not. The answer is 4. Only two of these calls return that number without reading the rows into script.

GlideQuery count(). This is the record count. where is green because the builder is still open. count is amber because it is the terminal. It returns a number. Nothing chains after it, including orElse and map.

```

var n = new GlideQuery('incident')
  .where('active', true)
  .count();
gs.info(n);
gs.info(typeof n);
// 4
// number

```

Stream. select returns the rows, not a count. toArray materializes them and length counts in script. reduce does the same walk without the array. Both log 4, and both read every active incident to do it. Use a Stream count only when you already needed the rows.

```

var rows = new GlideQuery('incident')
  .where('active', true)
  .select('number')
  .toArray(100);
gs.info(rows.length);
// 4

var n = new GlideQuery('incident')
  .where('active', true)
  .select('sys_id')
  .reduce(function (acc) { return acc + 1; }, 0);
gs.info(n);
// 4

```

Optional cannot count a set. It holds one value. selectOne collapses the query to a single row, so the only question it can answer is whether a row existed. isPresent logs true. Mapping that one row to 1 logs 1. Both are wrong as a count of 4. sum is the Optional that looks similar, and it is a total, not a length. count().orElse(0) does not compile into a chain: count already returned 4.

```

var one = new GlideQuery('incident')
  .where('active', true)
  .selectOne('number');
gs.info(one.isPresent());
// true, not 4

var asCount = one.map(function () { return 1; }).orElse(0);
gs.info(asCount);
// 1, wrong

// invalid: count() returned a number, orElse is an Optional call
// new GlideQuery('incident').where('active', true).count().orElse(0);

```

GlideRecord. getRowCount() is the comparison. It also returns a number, after query(). A bad field name is ignored, so this count can be every row. GlideQuery.count() would have thrown on that field.

```
var gr = new GlideRecord('incident');
gr.addQuery('active', true);
gr.query();
gs.info(gr.getRowCount());
// 4
```

Side by side on active = true. count() returns 4, a number. Stream toArray().length returns 4, after reading the rows. Stream reduce returns 4, after the same read. Optional selectOne mapped to 1 returns 1. GlideRecord getRowCount() returns 4.

Knowledge check

On the five sample rows, active incident count. What does each log? A. count() logs 4. B. selectOne().map(function(){return 1;}).orElse(0) logs 4. C. select().toArray(100).length logs 4. D. getRowCount() after addQuery active logs 4.

Answer. A, C, and D log 4. B logs 1. Optional held one row, so mapping it to 1 cannot see the other three.

Sources: ServiceNow GlideQuery API (Zurich developer reference and current scoped/global docs), the GlideQuery developer blog on data integrity and on there being no single-record delete, the Stream and Optional API pages, and the script-include alias of del to deleteMultiple. The method surface has changed across releases. If a name here is missing in a Background Script, inspect GlideQuery.prototype on that instance.